

PROJECT DOCUMENTATION

Embedded systems

Project Title:

Building Parking System

An Embedded Parking Management Solution

Field	Detail
Supervised By	Dr. Amr Fahmy
Course	Embedded systems design
Semester	Spring 2026
Academic Year	2025 – 2026

Table of Contents

1. Project Team	2
1.1 Team Members.....	2
1.2 Sub-Group Distribution	3
2. Introduction & Problem Statement	3
2.1 Background	3
2.2 Problem Statement	3
2.3 Motivation.....	4
3. Project Objectives	4
4. Technology Stack	4
5. System Architecture & Design.....	5
5.1 High-Level Architecture	5

5.2 Frontend Architecture (Web Dashboard).....	6
5.3 Backend Architecture (Firmware & Web Server)	7
5.4 Data Model	7
6. Project Steps & Phases	10
7. Group Task Distribution	11
8. Development Timeline	11
9. Full Project Documentation	12
9.1 Features & Functional Requirements.....	12
9.2 Non-Functional Requirements.....	13
9.3 API Documentation	13
9.4 UI / UX Design Notes	14
9.5 Testing Plan	14
9.6 Deployment & DevOps	15
10. Submission Checklist	15

1. Project Team

1.1 Team Members

The project was delivered by a four-member team. Each member owns an equal share of hardware wiring/testing, STM32 firmware, and documentation/defense, so that no individual is limited to a single repetitive task.

Full Name	College ID	Role / Responsibility	Email
Caroll	221004803	Slot Monitoring & Wireless Communication Specialist	Carollfroid331@gmail.com
Farida	221006595	Authentication & Structural Hardware Lead	Faridaahmed4006@gmail.com
Norhan	221006697	User Interface & System Integration Developer	norhanorabi3@gmail.com
Zeina	221004397	Gate Automation & Safety Systems Engineer	Zeina.moh.imam@gmail.com

1.2 Sub-Group Distribution

The system was divided into four module groups that mirror the layers of the application, from the embedded hardware up to the web dashboard.

Sub-Group	Member	Module / Layer	Key Deliverable
Web Dashboard & Comm.	Norhan	UI/UX, dashboard pages, live slot map, Node + MongoDB backend, LCD, ESP-01	Responsive dashboard with receipt & admin views
Firmware & Comm.	Caroll	Debounced slot polling & wrongpark logic, ESP-01 UART bridge, WiFiManager setup	Real-time slot telemetry over Wi-Fi
Access & Data	Farida	RFID authentication (user/VIP/admin/ride-share), card records, power & structure, Ultrasonic	Secure entry/exit access control
Gates & Safety / QA	Zeina	Servo gates, flame, buzzer, integration	Automated gates + emergency safety system

2. Introduction & Problem Statement

2.1 Background

Modern buildings operate with parking facilities with hundreds of spaces spread across multiple floors and zones. During peak hours, these lots fill quickly and drivers are left to search blindly for an empty space. The experience is slow and frustrating for residents and visitors and on a scale, this contributes to congestion, wasted fuel and avoidable emissions inside the structure.

Most existing building parking setups still rely on manual ticketing or a human operator at the barrier, and they offer no live view of which spaces are free. Management cannot see occupancy in real time, and visitors only discover a level is full after they have driven up to it. Crucially, conventional barriers are not tied into any safety system: in the event of a fire or other emergency the gates remain closed under manual control, so the parking cannot be evacuated quickly and vehicles can become trapped.

This project bridges embedded systems with web engineering. An STM32 microcontroller running FreeRTOS reads the parking hardware (slot sensors, RFID readers, ultrasonic and flame sensors) and an ESP-01 Wi-Fi module publishes that data to a connected web dashboard. The result is a smart, wireless building parking system that detects free slots in real time, automates secure entry and exit, protects against emergencies, and reports availability and receipts to any browser on the network.

2.2 Problem Statement

The system targets the following concrete problems found in conventional building parking:

- **Wasted time and congestion:** Drivers spend several minutes per visit hunting for an empty slot, especially at peak hours, creating queues and bottlenecks inside the lot.
- **No live availability information:** Neither drivers nor staff can see which spaces are free without physically checking each one, so there is no way to direct a car to an open slot.

- **Slow, manually operated entry and exit:** Ticket-based or operator-run barriers are slow, are prone to human error, and provide no record of who entered or how long they stayed.
- **Emergency evacuation risk:** In a fire or emergency the parking cannot be evacuated quickly manually controlled gates stay shut, there is no automatic alarm tied to the barriers, and cars and people can be trapped inside.
- **No visit record or receipt:** Drivers receive no clear summary of their stay (entry/exit time, slot, fee), and the facility has no automatic basis for charging penalties for misuse such as blocking the gate.

2.3 Motivation

Solving these problems matters because parking is the first and last impression a building makes on every tenant or visitor, and because safety in an enclosed, vehicle-filled structure is non-negotiable. A system that shows free spaces in real time saves drivers time and fuel and eases congestion; automated, authenticated gates make entry faster and accountable; and an integrated flame detection alarm that opens the barriers instantly can prevent vehicles and people from being trapped in an emergency. Built on affordable, widely available components, the design is realistic to scale from a demonstration maquette to a full multi-zone building deployment.

3. Project Objectives

By the end of the project, the team set out to achieve the following measurable goals:

- Build a real-time embedded parking system on an STM32 Blackpill running FreeRTOS that monitors every parking slot using IR sensors.
- Automate secure entry and exit using dual RC522 RFID readers on a shared SPI bus, admitting only registered cards and rejecting unauthorized ones.
- Drive the entry and exit barriers with servo motors triggered by ultrasonic detection of approaching vehicles.
- Guarantee anti-crush gate safety so the barrier never closes while an IR sensor beneath it detects a vehicle and applies penalty charges for cars that linger under the gate.
- Implement a flame-sensor fire-alarm subsystem that sounds an active buzzer and auto-opens all gates with minimal latency through a top-priority FreeRTOS safety task, holding the gates open until every sensor is clear for emergency evacuation.
- Assign each driver a specific free slot on the I2C LCD on entry and publish live availability plus an exit receipt to a connected web dashboard over the ESP-01 Wi-Fi module.
- Support four classes of RFID card — standard driver, VIP (reserved slot, no fee), admin (manual gate override) and ride-share/Uber (managed pass-through) — each with its own gate behaviour.
- Use a two-step entry (assign a slot, then confirm with a second scan to open the gate) and a twostep exit (compute the fee, then a payment scan releases the barrier) so the driver is always guided to the correct slot and billed correctly.
- Detect wrong parking (a car in an unassigned slot) and either re-map the occupancy automatically or raise an audible “slot overridden” warning, and alarm on any unauthorized car in the VIP slot.
- Forward every event as JSON over the ESP-01 to a Node.js backend that persists the data in MongoDB and serves the live dashboard, exit receipts and admin log.

- Demonstrate the complete, integrated system on a physical maquette (a scale model of a building parking lot).

4. Technology Stack

Every major technology, component and tool used across the embedded hardware, firmware and web layers of the project:

Layer	Technology / Tool	Purpose / Notes
Microcontroller	STM32 Blackpill (STM32F4)	Central controller running all real-time logic
RTOS	FreeRTOS	Parallel task scheduling (slot, access, display, safety)
Wireless Link	ESP-01 (ESP8266) over UART, 3.3 V	Forwards STM32 events over Wi-Fi to the backend (HTTP POST)
Frontend	HTML5, CSS3, JavaScript	Live dashboard: slot map, available-slots, receipt, admin
Backend / Database	Node.js web server + MongoDB (port 3000)	Ingests events at /api/event, stores sessions/logs and serves the dashboard
Wi-Fi Provisioning	WiFiManager captive portal + LittleFS	On-device setup hotspot; Wi-Fi and server URL saved to flash (no hardcoded credentials)
Slot Sensing	IR sensors (one per slot)	Detect occupied vs. available for each space
Access Control	Dual RC522 RFID readers (SPI)	Authenticate cards at entry and exit gates
Gate Actuation	Servo motors + HC-SR04 ultrasonic	Open barriers for approaching vehicles
Safety	Flame sensor + active buzzer	Fire detection, alarm and automatic gate opening
Layer	Technology / Tool	Purpose / Notes
Display	I2C LCD (PB6 / PB7)	Slot assignment and status shown to the driver
Communication Buses	UART, SPI, I2C	Inter-module data exchange on the STM32

Version Control	Git + GitHub	Source-code management and project README
IDE / Toolchain	STM32CubeIDE (HAL)	Firmware development, build and flashing
Power	Regulated 3.3 V and 5 V rails	Logic and peripheral power distribution

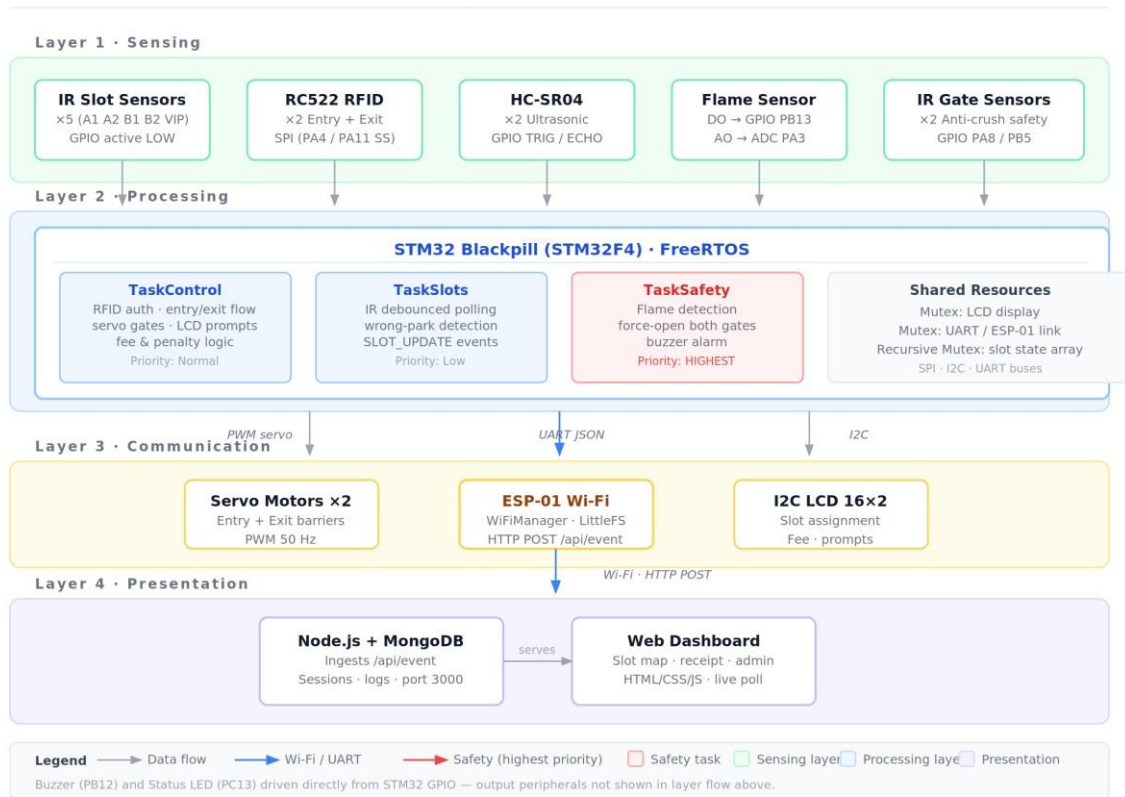
5. System Architecture & Design

5.1 High-Level Architecture

The system is organised into four cooperating layers. The sensing layer (IR slot sensors, RC522 RFID readers, HC-SR04 ultrasonic sensors and a flame sensor) captures the physical state of the lot. The processing layer the STM32 Blackpill running FreeRTOS reads every sensor, runs the control logic, drives the servo gates and buzzer, and updates the I2C LCD. The communication layer is the ESP-01 WiFi module, connected to the STM32 over UART; it receives one JSON event string per action and forwards each one over Wi-Fi to the backend with an HTTP POST. The presentation layer is a Node.js web server backed by MongoDB that ingests those events, stores sessions and logs, and serves the live dashboard to any browser on the network, together with the on-site LCD, which together inform drivers and administrators.

Data flows in a single, clear direction during normal operation: sensors → STM32 (FreeRTOS tasks) → ESP-01 → Node.js server + MongoDB → web dashboard, with the LCD branching directly off the STM32 for the driver at the gate. Safety events take priority over this flow: the top-priority safety task acts immediately on the gates and buzzer before any network update.

Smart Mall Parking System — Architecture



5.2 Frontend Architecture (Web Dashboard)

The dashboard is built with plain HTML, CSS and JavaScript so that it is lightweight enough to be served by the ESP-01. It avoids heavy frameworks and instead polls the ESP-01 endpoints at a short interval, redrawing slot tiles as their state changes.

- **Pages** — Home / live slot map, Available-slots summary, Exit receipt, and an Admin monitoring view.
- **State & updates** — Vanilla JavaScript fetches slot and gate telemetry on a timer; each slot tile is recolored (green = free, red = occupied) the moment an IR sensor state changes.
- **Styling** — A responsive, color-coded layout designed to be readable at a glance on both phones and desktop browsers.

5.3 Backend Architecture (Firmware & Web Server)

The STM32 firmware is structured around FreeRTOS, which splits the workload into independent parallel tasks so that one subsystem can never block another:

- **TaskControl (Farida + Norhan)** : The main interaction loop. It detects approaching vehicles with the HC-SR04 ultrasonics, reads card UIDs from the dual RC522 readers over SPI, runs the two-step entry (assign slot → confirm scan → open) and two-step exit (compute fee → payment scan → open) flows for standard, VIP, admin and ride-share cards, drives the servo gates, and updates the I2C LCD with prompts, slot assignments, fees and receipts.
- **TaskSlots (Caroll)** : Debounced polling of the IR slot array (A1, A2, B1, B2 and the VIP slot). It maintains the live available count, detects wrong parking (a car in an unassigned slot) and either

remaps the occupancy or flags an override, raises the VIP-intrusion alarm, and coalesces SLOT_UPDATE events sent to the ESP-01.

- **TaskSafety (Zeina):** The highest-priority task. It polls the flame sensor and, on detection, immediately forces both gates open, sounds the buzzer and holds the gates open until every IR sensor is clear of cars, re-arming if the flame returns before resetting the system to normal.
- **Concurrency (all members):** The three tasks run at priorities Safety > Control > Slots and share state safely through three mutexes: one for the LCD, one for the serial link to the ESP-01, and a recursive mutex that guards the parking-state arrays so no two tasks corrupt the slot data.

The ESP-01 does not host the dashboard; it acts as a Wi-Fi client that forwards each STM32 event as a JSON string via an HTTP POST to the backend at /api/event. The Node.js server stores the data in MongoDB and serves the slot map, receipt and admin views. The STM32 ↔ ESP-01 link is plain UART at 9600 baud, 3.3 V logic levels.

5.4 Data Model

The system maintains three logical entities, held partly in STM32 memory and partly in the dashboard's data store:

- **Registered Cards:** UID, owner reference and status (registered / blocked) used by the Access Control task to admit or reject a card.
- **Slots:** Slot ID, zone, current status (free / occupied) and last-update time the source for the live dashboard map.
- **Sessions / Tickets:** Card UID, assigned slot, entry time, exit time and computed fee or penalty the basis for the exit receipt.

5.5 Hardware Components and How They Communicate

Every peripheral in the system is wired directly to the STM32 Blackpill, which acts as the single master controller. The table below describes each component, the job it does, and the interface it uses to exchange data with the microcontroller.

Component	Description / Role	How it communicates with the STM32
STM32 Blackpill (STM32F4)	Central controller. Runs all FreeRTOS tasks, reads every sensor, drives the gates, buzzer and LCD, and talks to the ESP-01.	Master of all buses (SPI, I2C, UART) and every GPIO line.
MFRC522 RFID reader (x2)	Reads the card UID at the entry gate and the exit gate to authenticate drivers.	Shared SPI bus, each reader with its own chip-select (SS) line.
SG90 servo motor (x2)	Opens and closes the entry and exit barriers by rotating between the open and closed angles.	PWM on one GPIO each (50 Hz; pulse width ~1-2 ms sets the angle).
HC-SR04 ultrasonic (x2)	Detects a vehicle approaching each gate so the system prompts for an RFID scan.	Two GPIO lines: TRIG output pulse and ECHO input (distance = echo time).

IR slot sensor (x5)	One per space (A1, A2, B1, B2 and VIP). Detects whether the space is occupied.	Digital GPIO input, active LOW.
Component	Description / Role	How it communicates with the STM32
IR gate sensor (x2)	Under each barrier; provides anti-crush safety so a gate never closes on a car.	Digital GPIO input, active LOW.
Flame sensor	Detects fire and triggers the highest-priority routine that opens both gates.	Digital DO (active HIGH) on a GPIO, plus analog AO read by the STM32 ADC.
I2C 16x2 LCD	Shows driver prompts, the assigned slot, fees, penalties and warnings.	I2C bus (SDA + SCL) via a PCF8574 backpack.
Active buzzer	Sounds the fire alarm and signals penalties, wrong parking and denied access.	Digital GPIO output (HIGH = on).
On-board LED (PC13)	Visual status / blink feedback for confirmations and warnings.	Digital GPIO output (active LOW on the Blackpill).
ESP-01 (ESP8266)	Receives every event as a JSON string and forwards it over Wi-Fi (HTTP POST) to the Node.js + MongoDB backend that serves the dashboard.	UART (TX/RX) at 9600 baud, 3.3 V logic.

5.6 STM32 Pin Connection Map

The following map lists every signal exactly as assigned in the firmware. Pins are grouped by subsystem. "Direction" is from the point of view of the STM32 (OUT = the STM32 drives the line, IN = the STM32 reads it).

Subsystem	Signal	STM32 Pin	Dir.	Interface
RFID (SPI bus)	SPI clock (SCLK)	PA5	OUT	SPI
RFID (SPI bus)	SPI master-in (MISO)	PA6	IN	SPI
RFID (SPI bus)	SPI master-out (MOSI)	PA7	OUT	SPI
RFID (SPI bus)	Entry reader select (SS)	PA4	OUT	SPI chip-select
RFID (SPI bus)	Exit reader select (SS)	PA11	OUT	SPI chip-select
RFID (SPI bus)	Reader reset (RST)	Tied to 3.3 V	-	Hardwired high (RST_PIN = -1)

Gates (servos)	Entry barrier servo	PA0	OUT	PWM
Gates (servos)	Exit barrier servo	PA15	OUT	PWM
Vehicle detect	Entry ultrasonic TRIG	PB1	OUT	GPIO pulse
Vehicle detect	Entry ultrasonic ECHO	PB0	IN	GPIO timing
Vehicle detect	Exit ultrasonic TRIG	PA1	OUT	GPIO pulse
Vehicle detect	Exit ultrasonic ECHO	PA2	IN	GPIO timing
Subsystem	Signal	STM32 Pin	Dir.	Interface
Gate safety (IR)	Entry gate IR	PA8	IN	GPIO (active LOW)
Gate safety (IR)	Exit gate IR	PB5	IN	GPIO (active LOW)
Slot sensing (IR)	Slot A1	PB3	IN	GPIO (active LOW)
Slot sensing (IR)	Slot A2	PB4	IN	GPIO (active LOW)
Slot sensing (IR)	Slot B1	PB9	IN	GPIO (active LOW)
Slot sensing (IR)	Slot B2	PB8	IN	GPIO (active LOW)
Slot sensing (IR)	VIP slot	PA12	IN	GPIO (active LOW)
Fire safety	Flame digital out (DO)	PB13	IN	GPIO (active HIGH)
Fire safety	Flame analog out (AO)	PA3	IN	ADC (analog)
Display	LCD data (SDA)	PB7	I/O	I2C
Display	LCD clock (SCL)	PB6	OUT	I2C
Feedback	Buzzer	PB12	OUT	GPIO
Feedback	Status LED (on-board)	PC13	OUT	GPIO (active LOW)

Wireless link	UART TX to ESP-01	PA9	OUT	UART (9600)
Wireless link	UART RX from ESP-01	PA10	IN	UART (9600)

5.7 Communication Interfaces Summary

The STM32 acts as the master on three hardware buses and uses plain GPIO, PWM and the ADC for the remaining devices. This is how the pins above group into the communication channels of the system.

Interface	Pins used	Devices on it	How it works
SPI	PA5, PA6, PA7 (+ PA4, PA11 selects)	Both MFRC522 RFID readers	One shared synchronous bus; the STM32 selects one reader at a time with its SS line, then clocks the UID out.
I2C	PB6 (SCL), PB7 (SDA)	16x2 LCD (PCF8574 backpack)	Two-wire bus; the STM32 addresses the LCD backpack and sends characters and commands.
Interface	Pins used	Devices on it	How it works
UART	PA9 (TX), PA10 (RX)	ESP-01 Wi-Fi module	Asynchronous serial at 9600 baud; the STM32 sends one JSON event string per action to the ESP-01.
PWM	PA0, PA15	Entry and exit servos	A 50 Hz pulse-width signal; the pulse length commands the barrier angle (open / closed).
GPIO digital input	PA8, PB5, PB3, PB4, PB9, PB8, PA12, PB13	Gate IR, slot IR, VIP IR, flame DO	The STM32 reads each line as HIGH/LOW; IR sensors are active LOW, the flame DO is active HIGH.
GPIO timed I/O	PB1/PB0, PA1/PA2	HC-SR04 ultrasonic (x2)	The STM32 sends a short TRIG pulse and times the returning ECHO pulse to compute distance.
GPIO digital output	PB12, PC13	Buzzer, status LED	The STM32 drives the line HIGH/LOW to switch the buzzer or LED.
ADC (analog input)	PA3	Flame sensor analog out (AO)	The STM32 reads a varying voltage that reflects how strong the detected flame is.

6. Project Steps & Phases

Each development phase, its main tasks and the deliverable it produced:

Phase	Step / Task	Expected Output / Deliverable
Phase 1	Requirements gathering, component selection and system block diagram	Approved scope, bill of materials, block diagram
Phase 2	Simple breadboard prototype: STM32 + a single IR slot sensor with strict colour-coded rails	Working slot-detection circuit on one zone
Phase 3	Interface the ESP-01 Wi-Fi module over UART and establish connectivity	STM32 ↔ ESP-01 link; slot data published over Wi-Fi
Phase 4	Add RC522 RFID access, servo gates and ultrasonic detection	Authenticated entry/exit with automatic barriers
Phase 5	Build the web dashboard (HTML/CSS/JS): slot map, available slots, receipt, admin	Live dashboard reading hardware telemetry
Phase 6	Safety subsystem: flame sensor, buzzer, anticrush logic and penalty charging	Emergency auto-open and gate-safety logic
Phase 7	Integrate all modules under FreeRTOS as parallel tasks	Stable, responsive multi-task system
Phase	Step / Task	Expected Output / Deliverable
Phase 8	Assemble the maquette, full system testing and final demo	Complete working scale model + final report

7. Group Task Distribution

Each module is assigned to the member who owns it, with the workload kept balanced across hardware, firmware and presentation.

Module / Feature	Assigned Member	Sub-Group	Status
IR slot array + wrong-park detection	Caroll	Firmware & Comm.	Completed
ESP-01 Wi-Fi bridge & WiFiManager setup	Caroll	Firmware & Comm.	Completed
RFID access (user/VIP/admin/rideshare)	Farida	Access & Data	Completed
Breadboard layout & power rails	Farida	Access & Data	Completed

Dashboard + Node/MongoDB backend	Norhan	Web Dashboard	Completed
LCD prompts & slot-confirm flow	Norhan	Web Dashboard	Completed
Servo gates + ultrasonic detection	Zeina	Gates & Safety	Completed
Flame safety task, buzzer & VIPintrusion alarm	Zeina	Gates & Safety	Completed
FreeRTOS task architecture	All Members	Shared	Completed
GitHub repository & README	All Members	Shared	Completed

8. Development Timeline

Indicative weekly schedule (adjust to the actual course calendar):

Week	Phase / Task	Responsible	Deliverable
1	Requirements, component selection, block diagram	All	Scope + bill of materials
2	Breadboard prototype + IR slot detection	Caroll + Farida	Slot-detection circuit
3	ESP-01 integration and Wi-Fi link	Caroll	Telemetry over Wi-Fi
4	RFID access + servo / ultrasonic gates	Farida + Zeina	Automated entry/exit
5	Web dashboard + LCD slot assignment	Norhan	Live dashboard + LCD
6	Safety subsystem (flame, buzzer, anticrush, penalty)	Zeina	Emergency + safety logic
7	FreeRTOS integration and testing	All	Stable integrated system
Week	Phase / Task	Responsible	Deliverable
8	Maquette assembly, demo and report	All	Final model + documentation

9. Full Project Documentation

9.1 Features & Functional Requirements

The completed system supports the following features:

- **Real-time slot detection:** IR sensors monitor every space continuously and report it as free or occupied within seconds.

- **RFID access control:** Dual RC522 readers authenticate cards at the entry and exit gates; registered cards are admitted and unauthorized cards are rejected.
- **Automatic gate control:** Ultrasonic sensors detect an approaching vehicle and servo motors open the barrier without an operator.
- **Anti-crush gate safety:** The gate will not close while an IR sensor beneath it still detects a vehicle, preventing the barrier from closing on a car.
- **Penalty charging:** Vehicles that linger under the gate beyond a set threshold (5 s) are flagged and charged a penalty of 10 EGP, on top of the standard 5 EGP-per-minute parking fee, discouraging misuse of the entry/exit point.
- **Fire alarm & buzzer:** A flame sensor triggers the active buzzer and automatically opens all gates through the top-priority safety task, enabling immediate evacuation.
- **Live slots dashboard:** A web page maps every space and updates in real time (green = free, red = occupied) as the lot fills and empties.
- **Exit receipt:** After a car exits, the interface shows the driver a receipt summarizing the visit entry/exit time, slot and fee.
- **Wireless connectivity:** The ESP-01 module keeps the system online, so availability and receipts are reachable from any phone or computer on the network.
- **Admin Panel:** : A live web dashboard that displays real-time parking occupancy, physical gate positions, and chronological system logs for easy monitoring.
- **Admin Card Access:** A master RFID card that grants operators immediate manual override control to force any gate open or closed instantly.
- **VIP Card Access:** Special RFID authentication that guarantees entry to a reserved space even when the main lot is full, completely bypassing fees.
- **Ride-share / Uber access:** A managed pass-through card lets a ride-share vehicle open the entry or exit gate without being assigned a parking slot, for quick drop-off and pick-up.
- **Guided two-step entry & exit:** On entry the system assigns a free slot, shows it on the LCD and opens the gate only after the driver confirms with a second scan; on exit it shows the fee and penalty, then releases the barrier on a payment scan and prints a receipt.
- **Wrong-parking detection:** If a car parks in a slot it was not assigned, the system re-maps the occupancy to the correct driver where possible, or raises an audible “slot overridden” warning and logs it.
- **VIP-slot intrusion alarm:** If a non-VIP car occupies the reserved VIP slot, the buzzer sounds and the LCD warns the driver to leave until the slot is clear.

9.2 Non-Functional Requirements

- **Real-time performance:** Slot status is reflected on the dashboard within roughly one to two seconds; the flame-to-gate-open response is near-instant because the safety task runs at the highest priority and pre-empt all normal gate operations.
- **Safety:** Zero-latency emergency override and a guaranteed anti-crush rule take priority over all normal gate operations.
- **Reliability:** FreeRTOS isolates the tasks so a fault in one subsystem does not block the others; the RFID reset pin is pulled up to eliminate unstable reads.
- **Resource efficiency:** The two RFID readers share a single SPI bus with separate slave-select pins to save microcontroller pins, and the dashboard uses vanilla JavaScript to fit the ESP-01.

- **Power integrity:** Separate, regulated 3.3 V and 5 V rails with strict colour-coded wiring keep logic and peripherals stable.
- **Usability:** Clear LCD prompts at the gate and an intuitive, color-coded dashboard make the system easy to use for drivers and staff alike.

9.3 API Documentation

The system uses a single HTTP endpoint rather than a REST API. The STM32 emits one JSON event per action over UART; the ESP-01 forwards it unchanged to the backend with an HTTP POST. The Node.js server ingests the event, persists it in MongoDB and pushes the update to the dashboard.

POST /api/event — Content-Type: application/json. Returns 200/201 on success. The event types the STM32 can send:

Event	When it is sent
ENTRY	Driver confirms the assigned slot with a second scan; the entry gate opens.
EXIT_PENDING_PAYMENT	Exit scan; the parking fee and any penalty are computed, awaiting payment.
PAYMENT_DONE	Payment scan accepted; the exit gate opens and the slot is freed.
PAYMENT_DENIED	A different card was presented at the payment step.
PENALTY	A car blocked the entry gate beyond 5 s; a 10 EGP penalty is added.
VIP_ENTRY / VIP_EXIT	VIP card admitted to / released from the reserved slot (no fee).
UBER_ENTRY / UBER_EXIT	Ride-share pass-through opened at the entry or exit gate.
ADMIN_ACCESS	Admin master card used for a manual gate override.
UNKNOWN_CARD	An unregistered card was scanned and rejected.
ENTRY_DENIED_* / EXIT_DENIED_*	A card already inside tried to re-enter, or a card not inside tried to exit.
SLOT_UPDATE	Slot occupancy changed (sensor-driven); pushed to refresh the live map.
FIRE_ALARM / FIRE_CLEARED	Flame detected (gates forced open) / emergency ended and system reset.

JSON payload fields sent with every event:

Field	Meaning
event	The event name (one of the values in the table above).

uid	Card UID as an 8-character hex string, or “none” for sensor events.
fee	Parking fee in EGP (5 EGP per minute).
penalty	Penalty charge in EGP (10 EGP for blocking the gate).
duration	Minutes parked (sent on exit and VIP exit).
slotsLeft	Number of free standard slots remaining.
slotStates	Per-slot occupancy: A1, A2, B1, B2 and VIP (true = taken).

9.4 UI / UX Design Notes

- **Colour palette:** Primary navy , teal ,mint accent, available-green and occupied-red #E8584F for slot states.
- **Typography:** Clean sans-serif type (Segoe UI / Arial), generous line height, and large color-coded slot tiles for at-a-glance reading.
- **Driver entry flow:** Car detected → RFID scan → LCD shows the assigned slot → confirm with a second scan → gate opens → driver parks.
- **Driver exit flow:** RFID scan → fee/penalty computed → payment scan → gate opens → receipt displayed.
- **Admin flow:** Open the dashboard → monitor every zone live → review sessions and occupancy.

9.5 Testing Plan

Test Type	Tool / Method	Scope	Pass Criteria
Sensor unit test	Serial monitor / multimeter	IR, ultrasonic, flame, RFID	Correct reading for every input state
Integration test	Full hardware + dashboard	STM32 ↔ ESP-01 ↔ dashboard	Dashboard matches physical lot state
Access-control test	Registered & rejected cards	RFID entry / exit	Only valid cards are admitted
Emergency / fire test	Controlled flame source	Flame → buzzer → gate	Buzzer sounds and gates open under ~1 s
Anti-crush test	Object placed under gate	Gate-close logic	Gate refuses to close while occupied
Test Type	Tool / Method	Scope	Pass Criteria

Dashboard responsiveness	Browser (phone & desktop)	All UI pages	No layout breaks; live refresh works
--------------------------	---------------------------	--------------	--------------------------------------

9.6 Deployment & DevOps

- **Hosting:** A Node.js web server (port 3000) on a host PC on the local Wi-Fi network ingests events at /api/event, stores them in MongoDB, and serves the dashboard pages to any connected browser; the ESP-01 is a Wi-Fi client that forwards the STM32 events to it.
- **Firmware flashing:** STM32 firmware is built and flashed from STM32CubeIDE via an ST-Link programmer.
- **Version control:** Source is hosted on GitHub at github.com/Norhanoraby/parking-system, with a README documenting the STM32 configuration and wiring.
- **Configuration:** Wi-Fi is not hard-coded: on first boot the ESP-01 opens a WiFiManager captiveportal hotspot (“Parkly-Setup”) where the network and the backend server URL are entered and then saved to LittleFS flash, so they survive reboots; holding the FLASH button for ~3 s reopens the portal to change them. The registered-card UID list is set in the STM32 firmware, and power is supplied through regulated 3.3 V and 5 V rails.
- **Repository structure:** /firmware (STM32 + FreeRTOS), /web (dashboard), /hardware (wiring & pin maps) and /docs (report, manual, presentation).

10. GITHUB LINK

<https://github.com/Norhanoraby/parking-system>

11. Submission Checklist

--

- ✓ System block / architecture diagram completed
- ✓ Wiring diagram and pin map finalised
- ✓ IR slot-detection circuit tested across all zones
- ✓ ESP-01 Wi-Fi bridge and web server working
- ✓ RFID entry/exit access control verified
- ✓ Servo gates and ultrasonic detection functional
- ✓ Anti-crush gate safety validated
- ✓ Flame-sensor fire alarm and buzzer auto-open tested
- ✓ Penalty-charging logic implemented
- ✓ I2C LCD slot assignment working
- ✓ Web dashboard (slot map, available slots, receipt, admin) live
- ✓ FreeRTOS task architecture integrated and stable
- ✓ Maquette assembled and demonstrated

- ✓ Code pushed to GitHub with a descriptive README
- ✓ Final project report completed and formatted
- ✓ Presentation slides prepared
- ✓ Live demo rehearsed by all team members

Parkly Console

Online - Live Stream OK

Live Parking Layout Map

4 slots available

VIP BAY

READY

SLOT B2

VACANT

SLOT B1

VACANT

SLOT A2

VACANT

SLOT A1

VACANT

ENT

Parkly Console

Online - Live Stream OK

Live Parking Layout Map

3 slots available

VIP BAY

READY

SLOT B2

VACANT

SLOT B1

VACANT

SLOT A2

VACANT

SLOT A1

OCCUPIED

ENT

Parkly Console

Online - Live Stream OK

Live Parking Layout Map

2 slots available

VIP BAY

OCCUPIED

SLOT B2

VACANT

SLOT B1

OCCUPIED

SLOT A2

VACANT

SLOT A1

OCCUPIED

ENT

Parkly Console

Online - Live Stream OK

Admin Terminal

Back to Map

Full system event log

User / Card	Event	Fee	Slots Left	Time
farida (d19cd72a)	ADMIN_ACCESS	-	4 / 4	4:56:53 PM
farida (d19cd72a)	ADMIN_ACCESS	-	4 / 4	4:56:53 PM
farida (d19cd72a)	ADMIN_ACCESS	-	2 / 4	4:55:38 PM
farida (d19cd72a)	ADMIN_ACCESS	-	2 / 4	4:55:38 PM
Unknown (none)	SLOT_UPDATE	-	2 / 4	4:55:29 PM
Unknown (none)	SLOT_UPDATE	-	2 / 4	4:55:23 PM
Unknown (none)	SLOT_UPDATE	-	2 / 4	4:55:23 PM
vip card (d052d61)	VIP_ENTRY	-	2 / 4	4:55:18 PM
Unknown (none)	SLOT_UPDATE	-	2 / 4	4:55:07 PM
zeina (03109ba)	ENTRY	-	3 / 4	4:55:02 PM